

Deep Learning for Solving Dynamic Economic Models

Lilia Maliar, Serguei Maliar, Pablo Winant

presented by Jinxin and Jifan

November 29, 2022

- We want to approximate ("learn") an unknown function:

$$y = f(x)$$

where y is a scalar and $x = \{x_0, x_1, \dots, x_N\}$ a vector.

- A neural network is an approximation to $f(x)$ for

$$f(x) \approx g^{NN}(x; \theta) = \theta_0 + \sum_{m=1}^M \theta_m \phi\left(\sum_{n=0}^N \theta_{n,m} x_n\right)$$

- "Training" the network: select θ such that $g^{NN}(x; \theta)$ is as close to $f(x)$ as possible.
- A deep learning consists of layers of neutral networks.

This paper

- Deep learning method to solve dynamic economic models
- Cast three fundamental objects - *lifetime reward functions*, *Bellman equation* and *Euler equations* - into objective functions for Monte Carlo simulation.
- *All-in-one (AiO) expectation operator* for efficient approximation of integrals in Monte Carlo simulation
- A *stochastic gradient descent method* to train the three constructed objective functions
- Implement the DL solution framework using the DL platform Google TensorFlow

Casting Dynamic Economic Models into DL Expectation Functions

- An agent (consumer, firm, gov't, central bank, etc.) solves an intertemporal optimization problem.
- Given state (m_t, s_t) , an agent maximizes discounted lifetime reward

$$\max_{\{x_t, s_{t+1}\}_{t=0}^{\infty}} E_0 \left[\sum_{i=1}^{\infty} \beta^i r(m_t, s_t, x_t) \right]$$

st.

$$m_{t+1} = M(m_t, \epsilon_t) \quad (1)$$

$$s_{t+1} = S(m_t, s_t, x_t, m_{t+1}) \quad (2)$$

$$x_t \in X(m_t, s_t) \quad (3)$$

- Exogenous state m_{t+1} follows transition function M with i.i.d. shocks $\epsilon_t \in R^m$

Casting Dynamic Economic Models into DL Expectation Functions

- ① An optimal decision rule $x_t = \varphi(m_t, s_t) \in X(m_t, s_t)$ maximizes the lifetime reward for any initial conditions (m_0, s_0)
- ② A parametric decision rule is a function $\varphi(\cdot; \theta)$ parameterized by a real vector θ such that for each θ , we have φ and $x_t = \varphi(m_t, s_t)$ for all t .
- Goal: find a vector of parameters θ such that parametric decision rule $\varphi(\cdot; \theta)$ provides an accurate approx. of the optimal decision rule φ .

Objective functions for DL

Three fundamental objects of economic dynamics

- ① Lifetime reward of a consumer (to be maximized)
- ② Residuals in Euler equation (to be minimized)
- ③ Residuals in Bellman equation (to be minimized)

We show how to convert these three objects into the objective functions of DL algorithm.

Lifetime reward maximization- Typical

- For a given distribution of shocks $(\epsilon_1, \dots, \epsilon_T)$, $V(m_0, s_0)$ is a maximum expected lifetime reward from (m_0, s_0)

$$V(m_0, s_0) \equiv \max_{\{x_t, s_{t+1}\}_{t=0}^{\infty}} E_{(\epsilon_1, \dots, \epsilon_T)} \left[\sum_{t=0}^{\infty} \beta^t r(m_t, s_t, x_t) \right]$$

- To app. V , we replace inf. horizon problem by truncating it at T and simulate time series solution under a fixed decision rule $\varphi(\cdot; \theta)$

$$V(m_0, s_0; \theta)^T \equiv E_{(\epsilon_1, \dots, \epsilon_T)} \left[\sum_{t=0}^T \beta^t r(m_t, s_t, \varphi(m_t, s_t; \theta)) \right]$$

- Typical approach: Construct a solution on a grid of initial conditions $\{(m_0, s_0)\}$

Alternative approach

- Alternative approach: reformulate as an expectation function

$$\Xi(\theta) \equiv E_{\{m_0, s_0\}} \{ E_{(\epsilon_1, \dots, \epsilon_T)} \left[\sum_{t=0}^T \beta^t r(m_t, s_t, \varphi(m_t, s_t; \theta)) \right] \}$$

- $\Xi(\theta)$ has two types of randomness: future shocks $(\epsilon_1, \dots, \epsilon_T)$ and random state (m_0, s_0) . Costly: $n \times n'$ draws
- Solution: All-in-One (AiO) expectation operator: the dist. of the composite random variable $\omega \equiv (m_0, s_0, \dots, \epsilon_T)$.
- Lifetime reward becomes

$$\Xi(\theta) = E_{\omega} [\xi(\omega; \theta)] \equiv E_{\{m_0, s_0\}} \{ E_{(\epsilon_1, \dots, \epsilon_T)} \left[\sum_{t=0}^T \beta^t r(m_t, s_t, \varphi(m_t, s_t; \theta)) \right] \}$$

Euler residual minimization

- Set of equations

$$E_{\epsilon}[f_j(m, s, x, m', s', x')] = 0, j = 1, \dots, J$$

$$\text{st. } m' = M(m, \epsilon), s' = S(m, s, x, m') \quad \text{and} \quad x \in X(m, s)$$

- Typical approach: solve the EE on a fixed grid of the state space
- Alternative: the objective function is an expected squared sum of residuals in the EE for a given dist. of states.

$$\Xi(\theta) = E_{(m,s)} \left\{ \sum_{j=1}^J v_j (E_{\epsilon}[f_j(m, s, \varphi(m, s, ; \theta), m', s', \varphi(m', s'; \theta))])^2 \right\}$$

where $(v_1, \dots, v_J)$ is a vector of weights on J optimality conditions.

- Again, two expectation operators E_{ϵ} and $E_{(m,s)}$. Curse of dimensionality.

- AiO operator:

$$E_{\epsilon_1}[f(\epsilon_1)]E_{\epsilon_2}[f(\epsilon_2)] = E_{(\epsilon_1, \epsilon_2)}[f(\epsilon_1)f(\epsilon_2)]$$

- Euler-residual minimization with AiO expectation operator

$$\Xi_{(\theta)} = E_{\omega}[\xi(w; \theta)]$$

$$\equiv E_{(m, s, \epsilon_1, \epsilon_2)}\{\sum_{j=1}^J v_j[f_j(m, s, x, m', s', x') |_{\epsilon=\epsilon_1}][f_j(m, s, x, m', s', x') |_{\epsilon=\epsilon_2}]\}$$

- Construct $\varphi(., \theta)$ that solves $\min_{\theta \in \Theta} \Xi(\theta)$

Bellman residual minimization

- Parametrize a value function $V(., \theta_1)$, decision rule $x = \varphi(., \theta_2)$ and random variable $\omega \equiv (m, s)$, Bellman residual min. is

$$\Xi_{\theta} \equiv E_{(m,s)} \{ V(m, x; \theta_1) - r(m, s, x) - \beta E_{\epsilon} [V(m', s'; \theta_1)] \}^2 \\ + v E_{(m,s)} \{ V(m, x; \theta_1) - r_x(m, s, x) - \beta \{ E_{\epsilon} [V_{s'}(m', s'; \theta_1)] \} \frac{\partial s'}{\partial x} \}^2$$

- Bellman residual min. with AiO

$$\Xi(\theta) = E_{\omega} [\xi(\omega; \theta)] \equiv E_{(m,s,\epsilon_1,\epsilon_2)} \left\{ \left[V(m, s; \theta_1) - r(m, s, x) - \beta V(m', s'; \theta_1) \right]_{\epsilon=\epsilon_1} \right. \\ \times \left[V(m, s; \theta_1) - r(m, s, x) - \beta V(m', s'; \theta_1) \right]_{\epsilon=\epsilon_2} \\ \left. + v \left[r_x(m, s, x) + \beta V_{s'}(m', s'; \theta_1) \right]_{\epsilon=\epsilon_1} \frac{\partial s'}{\partial x} \left[r_x(m, s, x) + \beta V_{s'}(m', s'; \theta_1) \right]_{\epsilon=\epsilon_2} \frac{\partial s'}{\partial x} \right\}$$

Deep learning solution method

- Recast economic model as a problem of minimizing an obj. function $\Xi(\theta)$ w.r.t. a vector of parameters θ :

$$\min_{\theta \in \Theta} \Xi(\theta) = \min_{\theta \in \Theta} E_{\omega}[\xi(\omega; \theta)]$$

where $\omega \equiv (m, s, \epsilon)$

- Interpret not as a computational problem, but as an estimation/regression model.
- Simulate model to produce a set of random draws $\{\omega_i\}_{i=1}^n$, replace expected risk $\Xi(\theta)$ with empirical risk $\Xi^n(\theta)$

$$\min_{\theta \in \Theta} \Xi^n(\theta) = \min_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^n \xi(\omega_i; \theta)$$

- Then, construct $\hat{\theta}$ to minimize empirical risk on simulated data

DL algorithm for solving dynamic economic models

Step 1: Initialize the algorithm

- i). construct theoretical risk $\Xi(\theta) = E_{\omega}[\xi(\omega; \theta)]$
- ii). define empirical risk $\Xi^n(\theta) = \frac{1}{n} \sum_{i=1}^n \xi(\omega_i; \theta)$
- iii). define a topology of neural network $\varphi(., \theta)$
- iv). fix initial vector of the coeff. θ

Step 2: Train the machine, i.e., find θ that min. the empirical risk $\Xi^n(\theta)$

- i). Simulate model to produce data $\{\omega_i\}_{i=1}^n$ by decision rule $\varphi(., \theta)$
- ii). Construct gradient $\nabla \Xi^n(\theta) = \frac{1}{n} \sum_{i=1}^n \nabla \xi(\omega_i; \theta)$;
- iii). Update $\hat{\theta} = \theta - \lambda_k \nabla \Xi^n(\theta)$ and go to step 2.i); Stop if $\|\hat{\theta} - \theta\| < \epsilon$ is satisfied.

Step3: Assess the accuracy of the approx. $\varphi(., \theta)$ on a new sample

Numerical analysis of Krusell and Smith's (1998) model

- The economy consists of a set of heterogeneous agents $i=1, \dots, \ell$

$$\begin{aligned} \max_{\{c_t^i, k_{t+1}^i\}_{t=0}^{\infty}} E_0 \left[\sum_{t=0}^{\infty} \beta^t u(c_t^i) \right] \\ \text{s.t. } w_{t+1}^i = R_{t+1} (w_t^i - c_t^i) + W_{t+1} \exp(y_{t+1}^i), \\ c_t^i \leq w_t^i \end{aligned}$$

- c_t^i : consumption;
 w_t^i : cash-on-hand;
 y_t^i : labor productivity;
 R_t : interest rate;
 W_t : wage;
 $k_{t+1}^i = w_t^i - c_t^i$: next period capital;
Initial condition (y_0^i, w_0^i) is given.

Numerical analysis of Krusell and Smith's (1998) model

- The individual productivity evolves as

$$y_{t+1}^i = \rho_y y_t^i + \sigma_y \epsilon_t^i \text{ with } \epsilon_t^i \sim \mathcal{N}(0, 1)$$

- z_t is an aggregate productivity shock,

$$z_{t+1} = \rho z_t + \sigma \epsilon_t \text{ with } \epsilon_t \sim \mathcal{N}(0, 1).$$

- The equilibrium prices are

$$R_t = 1 - d + z_t \alpha k_t^{\alpha-1} \left[\sum_{i=1}^{\ell} \exp(y_t^i) \right]; \quad W_t = z_t (1 - \alpha) k_t^{\alpha} \left[\sum_{i=1}^{\ell} \exp(y_t^i) \right]$$

Deep learning solution algorithm

- State space: $\{y_t^i, w_t^i\}_{i=1}^\ell$, the aggregate shock z_t .
- Parameterization:

$$\begin{aligned}\frac{c_t^i}{w_t^i} &= \sigma(\zeta_0 + \eta(y_t^i, w_t^i, D_t, z_t; \vartheta)) \equiv \varphi(\cdot; \theta), \\ h_t^i &= \exp(\zeta_0 + \eta(y_t^i, w_t^i, D_t, z_t; \vartheta)) \equiv h(\cdot; \theta), \\ V_t^i &= \zeta_0 + \eta(y_t^i, w_t^i, D_t, z_t; \vartheta) \equiv V(\cdot; \theta),\end{aligned}$$

- $\eta(\cdot; \vartheta)$: neural network,
- $D_t \equiv \{y_t^i, w_t^i\}_{i=1}^\ell$: the distribution,
- $\theta \equiv (\zeta_0, \vartheta)$
- $\sigma(x) = \frac{1}{1+e^{-x}}$
- $\varphi(\cdot; \theta)$: sigmoid transformation

Deep learning solution algorithm

- Simulation. For the given economy's state $\left(\{w^i, y^i\}_{i=1}^{\ell}, z\right)$ and neural network coefficients $\theta \equiv (\zeta_0, \vartheta)$:
 - 1. Compute $\frac{c_t^i}{w_t^i} = \varphi(y_t^i, w_t^i, D_t, z_t; \theta)$ and $k_{t+1}^i = w_t - c_t$ for each agent $i = 1, \dots, \ell$.
 - 2. Draw y_{t+1}^i for $i = 1, \dots, \ell$ and z_{t+1}
 - 3. Compute prices R_{t+1} and W_{t+1} given $k_{t+1} = \sum_{i=1}^{\ell} k_{t+1}^i$.

- 4. Compute next period cash-in-hand
$$w_{t+1}^i = R_{t+1}k_{t+1}^i + W_{t+1} \exp(y_{t+1}^i).$$
- 5. Compute $\frac{c_{t+1}^i}{w_{t+1}^i} = \varphi(y_{t+1}^i, w_{t+1}^i, D_{t+1}, z_{t+1}; \theta)$ for $i = 1, \dots, \ell$.
- 6. Evaluate the objective function (lifetime reward, Euler and Bellman residuals), train the neural networks and go to next iteration.

Objective Function

- Lifetime reward:

$$\Xi(\theta) = E_{\omega}[\xi(\omega; \theta)] \equiv E_{(Y_0, W_0, Z_0, \Sigma, \epsilon)} \left[\sum_{t=0}^T \beta^t u(c_t) \right]$$

- Euler-equation method:

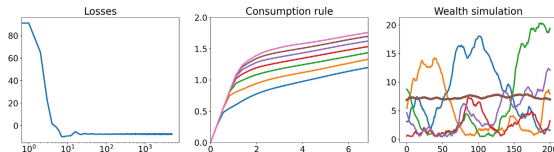
$$\begin{aligned} \Xi(\theta) = E_{\omega}[\xi(\omega; \theta)] = E_{(Y_t, W_t, Z_t, \Sigma_1, \Sigma_2, \epsilon_1, \epsilon_2)} & \left\{ \left[\Psi^{FB} \left(1 - \frac{c_t^i}{w_t^i}, 1 - h_t^i \right) \right]^2 \right. \\ & \left. + v \left[\frac{\beta R_{t+1} u'(c_{t+1}^i) |_{\Sigma=\Sigma_1, \epsilon=\epsilon_1}}{u'(c_t^i)} - h_t^i \right] \left[\frac{\beta R_{t+1} u'(c_{t+1}^i) |_{\Sigma=\Sigma_2, \epsilon=\epsilon_2}}{u'(c_t^i)} - h_t^i \right] \right\} \end{aligned}$$

- Bellman equation

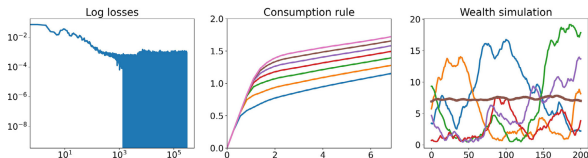
$$\begin{aligned} \Xi(\theta) = E_{\omega}[\xi(\omega; \theta)] \equiv E_{(Y_t, W_t, z_t, \Sigma_1, \Sigma_2, \epsilon_1, \epsilon_2)} & \left\{ \left[V(s_t^i; \theta) - u(c_t^i) - \beta V(s_{t+1}^i; \theta) \right]_{\Sigma=\Sigma_1, \epsilon=\epsilon_1} \right. \\ & \times \left[V(s_t^i; \theta) - u(c_t^i) - \beta V(s_{t+1}^i; \theta) \right]_{\Sigma=\Sigma_2, \epsilon=\epsilon_2} + v \left[\Psi^{FB} \left(1 - \frac{c_t^i}{w_t^i}, 1 - h_t^i \right) \right]^2 \\ & \left. + v_h \left[\frac{\beta \frac{\partial}{\partial w_{t+1}^i} V(s_{t+1}^i; \theta)}{u'(c_t^i)} \right]_{\Sigma=\Sigma_1, \epsilon=\epsilon_1} - h_t^i \right] \left[\frac{\beta \frac{\partial}{\partial w_{t+1}^i} V(s_{t+1}^i; \theta)}{u'(c_t^i)} \right]_{\Sigma=\Sigma_2, \epsilon=\epsilon_2} - h_t^i \right\} \end{aligned}$$

Objective Function

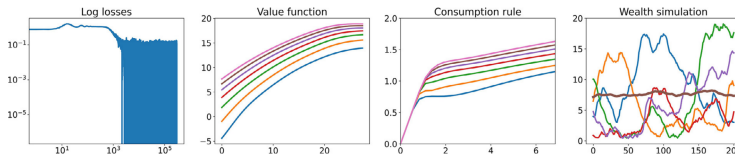
- Lifetime reward



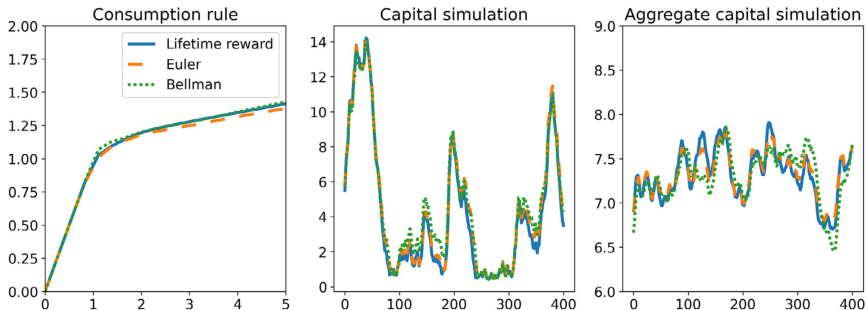
- Euler-equation method



- Bellman equation

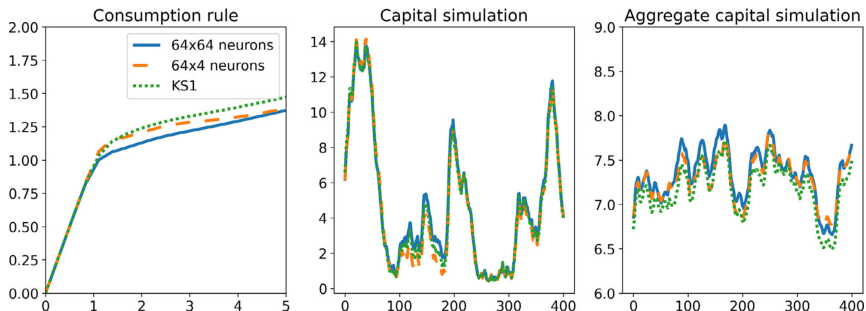


Comparison of the solutions produced by three methods



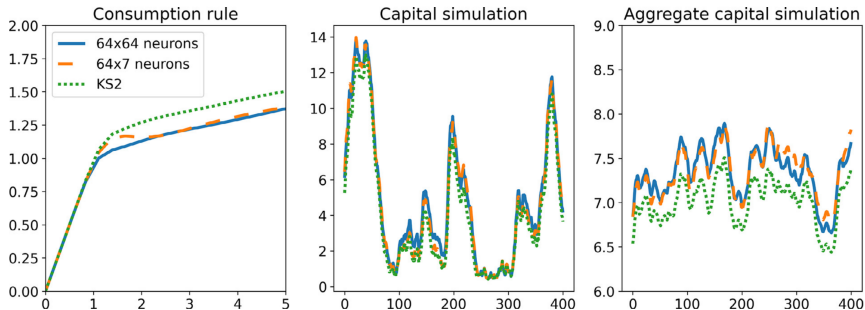
- The constructed decision rules and time series solutions are visually close

Deep learning versus moments



- The one moment KS1 solution is systematically shifted up relative to our baseline 64×64 solution.
 - The 64×4 -neuron solution is also shifted up near the kink but it gradually approaches our reference solution for larger levels of capital.
- $\eta(y_t^i, w_t^i, D_t, z_t; \vartheta)$

Deep learning versus moments



- We next consider a solution with seven neurons in the hidden layer 64×7 which has the same dimensionality as Krusell and Smith (1998) solution with first and second moments KS2. $\eta(y_t^i, w_t^i, D_t, z_t; \vartheta)$
- Additional neurons do help get closer to the reference solution
- This is because the second moments do not have additional explanatory power relative to the first moment

Conclusion and Discussion

- This paper proposes a deep learning method based on Monte Carlo simulation.
- The main novelty: computational technology. TensorFlow and Pytorch
- Neural networks are powerful, but expensive and their convergence to the solutions is not guaranteed.
- Learning is always deep in economic models.
- Data are truly random in economic dynamics.
- fixed-point iteration, conventional GD methods, and linear programming unsupervised and reinforcement learning methods.

Thank you